

ML ACADEMY

Python for ML & AI Interview Handbook

Detailed Questions, Explanations & Worked Examples for Top Tech Interviews

Python Fundamentals • Data Structures • NumPy • pandas • Coding Problems Algorithms • Complexity • ML Data Tasks • Debugging • Clean Interview Code

www.learnmlacademy.com

Free Resource - Share with your network

How to get the most from this handbook

This PDF is designed for serious interview preparation. Each topic combines an interview-ready answer with deeper explanation, examples, diagrams, code, calculations or system trade-offs wherever they improve understanding.

Recommended approach: answer the question yourself first, then compare with the explanation and work through the example.

For coding and SQL questions, state assumptions, approach, complexity and edge cases before writing code. For ML/AI system design, discuss requirements, trade-offs, failure modes, monitoring and security instead of claiming one architecture is always best.

Section	Topics
1. Research basis and further reading	1
TOTAL COVERAGE	1

1. Core array, stack and window patterns

These patterns test whether you can maintain a simple invariant under pressure.

Coding interview contract Before coding: clarify input/output, duplicates, ordering, invalid values and scale. During coding: explain the invariant. After coding: test a normal case, boundary case, complexity and a follow-up variant.

SOLVED PROBLEM2026 SIGNAL

Merge-sorted-list variants remain a standard big-tech coding pattern for MLE interviews.

Merge two sorted lists

Problem. Given two ascending lists, return one ascending list without calling sort on the combined data.

Reasoning. Use two pointers. At every step, the smallest unmerged element must be at one of the two current pointers. Append it and advance only that pointer.

Dry run a=[1,4,7], b=[2,3,8]. Compare 1 vs 2 -> take 1. Compare 4 vs 2 -> take 2. Compare 4 vs 3 -> take 3. Compare 4 vs 8 -> take 4. Compare 7 vs 8 -> take 7. Append remaining 8.

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i])
            i += 1
        else:
            out.append(b[j])
            j += 1
    out.extend(a[i:])
    out.extend(b[j:])
    return out

print(merge_sorted([1, 4, 7], [2, 3, 8]))
# [1, 2, 3, 4, 7, 8]
```

Complexity	Edge cases	Follow-up
O(n+m) time; O(n+m) output space.	Empty list, duplicates, one list entirely smaller, incomparable element types.	How would you merge k sorted lists? Use a min-heap of current heads for O(N log k).

SOLVED PROBLEM2026 SIGNAL

Bracket-validation variants appear in Meta/Amazon/Apple-style coding banks.

Validate balanced brackets

Problem. Return True when (), [], and {} are correctly balanced and nested.

Reasoning. A stack holds unmatched opening brackets. Each closing bracket must match the most recent opening bracket; if not, nesting is invalid.

Dry run '(){}': push (, push [, see) -> top is [, expected (. Return False immediately.

```
def valid_brackets(text):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in text:
        if ch in '([{':
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(valid_brackets('{[()]}')) # True
print(valid_brackets('([)]')) # False
```

Complexity	Edge cases	Follow-up
O(n) time; O(n) worst-case stack space.	Empty string, non-bracket characters, closing bracket first, unfinished openers.	Return the index and expected bracket for the first mismatch.

SOLVED PROBLEM

Sliding-window moving average

Problem. Compute every window mean for window size k without summing each window from scratch.

Reasoning. Keep a running sum. When the window moves right, add the entering value and subtract the leaving value.

Dry run[2,4] sum=6 -> 3. Move: +8 -2 =12 -> 6. Move: +10 -4=18 -> 9.

```
def moving_average(values, k):
    if k <= 0 or k > len(values):
        raise ValueError("invalid window")
    total = sum(values[:k])
    out = [total / k]
    for right in range(k, len(values)):
        total += values[right] - values[right-k]
        out.append(total / k)
    return out

print(moving_average([2, 4, 8, 10], 2))
# [3.0, 6.0, 9.0]
```

Complexity	Edge cases	Follow-up
O(n) time; O(n-k+1) output space.	k=1, k=n, invalid k, time gaps if the question means time windows rather than row windows.	Adapt it to an iterator when you cannot index the full input.

2. Trees, graphs, heaps and streaming data

MLE coding still includes conventional data structures. Be able to explain why the invariant makes the algorithm correct.

SOLVED PROBLEM[2026 SIGNAL](#)

LCA/shared-ancestor problems appear in current Google, Meta and Apple MLE question banks.

Lowest common ancestor in a binary tree

Problem. Given a binary tree and two node values, return their lowest common ancestor.

Reasoning. Recursively ask left and right subtrees. If the current node is a target, return it. If both sides return a non-null node, current is the lowest point where paths meet.

Dry runIf target a is found in the left subtree and b in the right subtree, both recursive calls return non-null at the current node, so current is their first shared ancestor.

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right

def lca(root, a, b):
    if root is None or root.value in (a, b):
        return root
    left = lca(root.left, a, b)
    right = lca(root.right, a, b)
    if left and right:
        return root
    return left or right
```

Complexity	Edge cases	Follow-up
$O(n)$ time; $O(h)$ recursion stack for tree height h .	Clarify whether both nodes exist and values are unique. A skewed tree gives $O(n)$ recursion depth.	If parent pointers are available, collect ancestors of one node and walk the other upward.

SOLVED PROBLEM[2026 SIGNAL](#)

Graph traversal remains a standard algorithmic expectation in MLE loops.

Shortest path in an unweighted graph

Problem. Return the minimum number of edges from start to target.

Reasoning. Breadth-first search explores vertices in layers of increasing distance. The first time target is dequeued, its distance is minimal in an unweighted graph.

Dry runQueue starts $[(A,0)]$. Visit all A neighbors at distance 1, then all their unseen neighbors at distance 2. This layer order is why BFS is shortest for unit edges.

```

from collections import deque

def shortest_path(graph, start, target):
    q = deque([(start, 0)])
    seen = {start}
    while q:
        node, dist = q.popleft()
        if node == target:
            return dist
        for nxt in graph.get(node, []):
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, dist + 1))
    return None

```

Complexity	Edge cases	Follow-up
$O(V+E)$ time; $O(V)$ space.	Cycles, disconnected graph, directed vs undirected semantics, $start==target$.	If edges have non-negative weights, use Dijkstra with a priority queue.

SOLVED PROBLEM2026 SIGNAL

Top-k streaming patterns are common in ranking, monitoring and big-tech coding interviews.

Top k values from a stream

Problem. Keep the k largest values seen so far without storing the entire stream.

Reasoning. Maintain a min-heap of size k. The heap root is the smallest value among the current top k. Replace it only when a new value is larger.

Dry run=3. Heap after 9,1,4 -> [1,9,4]. Read 12: replace 1 -> [4,9,12]. Read 3: ignore. Read 8: replace 4 -> [8,12,9].

```

import heapq

def top_k_stream(values, k):
    if k <= 0:
        return []
    heap = []
    for x in values:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)
    return sorted(heap, reverse=True)

print(top_k_stream([9, 1, 4, 12, 3, 8], 3))
# [12, 9, 8]

```

Complexity	Edge cases	Follow-up
$O(n \log k)$ time; $O(k)$ space.	$k \leq 0$, k greater than item count, duplicates, whether values must be distinct.	Return k most frequent values instead - now you need frequency counts plus heap/bucket strategy.

SOLVED PROBLEM2026 SIGNAL

Streaming-median problems test heap fluency and online-data reasoning.

Streaming median

Problem. Add numbers one by one and return the median after each insertion.

Reasoning. Use a max-heap for the lower half and a min-heap for the upper half. Keep sizes within one and ensure every lower-half value is $\leq$ every upper-half value.

Dry run Values 5,1,9,2. After balancing: low=[1,5] conceptually and high=[9] after 3 values -> median 5. Add 2 -> lower {1,2}, upper {5,9} -> median $(2+5)/2=3.5$.

```
import heapq

class MedianStream:
    def __init__(self):
        self.low = [] # negative values -> max heap
        self.high = []

    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def median(self):
        if not self.low:
            raise ValueError("empty")
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2
```

Complexity	Edge cases	Follow-up
$O(\log n)$ insertion; $O(1)$ median lookup; $O(n)$ stored values.	Empty stream, duplicates, negative values, precision for large numeric types.	How would you approximate a median over billions of distributed values? Discuss sketches/quantiles.

3. ML/data-flavored coding exercises

Role-specific exercises reward clean handling of metrics, vectors, batching and data-pipeline invariants.

SOLVED PROBLEM

Precision and recall from binary labels

Problem. Given `y_true` and `y_pred`, calculate precision and recall for positive label 1.

Reasoning. Count TP, FP and FN in one pass. Protect zero denominators explicitly.

Dry run `truth=[1,1,0,0], pred=[1,0,1,0] -> TP=1, FP=1, FN=1 -> precision=0.5, recall=0.5.`

```
def precision_recall(y_true, y_pred):
    if len(y_true) != len(y_pred):
        raise ValueError("length mismatch")
    tp = fp = fn = 0
    for truth, pred in zip(y_true, y_pred):
        tp += truth == 1 and pred == 1
        fp += truth == 0 and pred == 1
        fn += truth == 1 and pred == 0
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    return precision, recall
```

Complexity	Edge cases	Follow-up
O(n) time; O(1) auxiliary space.	Length mismatch, non-binary labels, no predicted positives, no actual positives.	Add F1 and multi-class macro/micro averaging.

SOLVED PROBLEM

Cosine similarity

Problem. Compute cosine similarity between two equal-length numeric vectors.

Reasoning. Cosine similarity is $\text{dot}(a,b)/(\|a\| \|b\|)$. Compute dot product and norms in one pass; a zero vector has undefined direction.

Dry run `[1,1] dot [1,0] = 1. Norms sqrt(2) and 1 -> similarity 1/sqrt(2) ≈ 0.707.`

```
from math import sqrt

def cosine(a, b):
    if len(a) != len(b):
        raise ValueError("length mismatch")
    dot = sum(x*y for x, y in zip(a, b))
    na = sqrt(sum(x*x for x in a))
    nb = sqrt(sum(y*y for y in b))
    if na == 0 or nb == 0:
        raise ValueError("zero vector")
    return dot / (na * nb)

print(round(cosine([1, 1], [1, 0]), 3)) # 0.707
```

Complexity	Edge cases	Follow-up
$O(d)$ time; $O(1)$ auxiliary space.	Zero vector, length mismatch, numeric overflow/precision, sparse high-dimensional vectors.	How does cosine differ from dot product when vectors are normalized?

SOLVED PROBLEM

Mini-batch generator

Problem. Yield consecutive mini-batches of size `batch_size` without constructing all batches at once.

Reasoning. A generator yields one slice at a time. Clarify whether to keep the final partial batch and whether shuffling is required.

Dry run`n=7,k=3` -> starts 0,3,6. The final slice naturally returns one element.

```
def batches(items, batch_size):
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    for start in range(0, len(items), batch_size):
        yield items[start:start + batch_size]

for batch in batches(list(range(7)), 3):
    print(batch)
# [0,1,2], [3,4,5], [6]
```

Complexity	Edge cases	Follow-up
$O(n)$ total time; $O(\text{batch_size})$ for each yielded slice.	Empty input, <code>batch_size > n</code> , final partial batch, input may be a one-pass iterator.	Rewrite for an iterator that has no <code>len()</code> or slicing.

SOLVED PROBLEM

Deduplicate events while preserving first-seen order

Problem. Keep the first record for each `event_id` and report duplicates.

Reasoning. Use a set for membership and append unseen records in arrival order. Define how to handle conflicting duplicate payloads.

Dry run`IDs e1,e1,e2` -> first `e1` stored; second `e1` added to duplicate list; `e2` stored.

```
def deduplicate(events):
    seen = set()
    clean = []
    duplicate_ids = []
    for event in events:
        event_id = event["event_id"]
        if event_id in seen:
            duplicate_ids.append(event_id)
            continue
        seen.add(event_id)
        clean.append(event)
    return clean, duplicate_ids
```

Complexity	Edge cases	Follow-up
Expected $O(n)$ time; $O(u)$ space for u unique IDs.	Missing/null ID, unhashable keys, conflicting duplicates, unbounded memory in an infinite stream.	Deduplicate only within a seven-day streaming window.

4. Practical Python design and ML pipeline reasoning

Interviewers often care more about edge cases, contracts and data leakage than clever syntax.

SOLVED PROBLEM

LRU cache

Problem. Implement get and put for a fixed-capacity least-recently-used cache in $O(1)$ average time.

Reasoning. Combine a hash map for key lookup with a doubly linked list for recency order. Python's `OrderedDict` can express the same design concisely if the interviewer allows standard-library containers.

Dry run capacity=2: put A, put B -> [A,B]. get A -> [B,A]. put C -> evict B -> [A,C].

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return None
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        if key in self.data:
            self.data.move_to_end(key)
        self.data[key] = value
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

Complexity	Edge cases	Follow-up
$O(1)$ average get/put; $O(\text{capacity})$ space.	Capacity zero, update existing key, missing-key convention, thread safety.	Implement without <code>OrderedDict</code> using a custom doubly linked list + dictionary.

SOLVED PROBLEM

Group records by key while preserving arrival order

Problem. Group dictionaries by `user_id` and keep events in original order inside each group.

Reasoning. A dictionary of lists performs one-pass grouping. In modern Python dictionaries preserve insertion order, but the problem only requires the lists to preserve record order.

Dry run As each record arrives, append it to exactly one user's list; no sorting is needed.

```

from collections import defaultdict

def group_by(records, key):
    groups = defaultdict(list)
    for record in records:
        if key not in record:
            raise KeyError(f"missing {key}")
        groups[record[key]].append(record)
    return dict(groups)

```

Complexity	Edge cases	Follow-up
Expected $O(n)$ time; $O(n)$ output space.	Missing key, unhashable key, copy vs reference semantics, very large groups.	Return only the latest event per user using timestamp comparison.

SOLVED PROBLEM

Min-max normalization without leakage

Problem. Write normalization code, then explain how it must be used correctly in an ML pipeline.

Reasoning. The coding part is straightforward, but the interview insight is that min/max must be fitted on training data and reused on validation/test data. Fitting on the full dataset leaks test-distribution information.

Dry run Train [2,4,6] -> low=2, high=6. Validation value 8 transforms to 1.5. That is acceptable: transform does not have to clamp to [0,1].

```

class MinMaxScalerSimple:
    def fit(self, values):
        if not values:
            raise ValueError("empty")
        self.low = min(values)
        self.high = max(values)
        return self

    def transform(self, values):
        if self.high == self.low:
            return [0.0 for _ in values]
        return [(x - self.low)/(self.high-self.low) for x in values]

scaler = MinMaxScalerSimple().fit(train_values)
val_scaled = scaler.transform(val_values)

```

Complexity	Edge cases	Follow-up
fit $O(n)$, transform $O(n)$; stores only low/high.	All equal values, new validation values outside training range, missing/non-finite values.	How would RobustScaler behave differently under extreme outliers?

Deep-dive Python interview tutorial: patterns, complexity and ML utilities

The solved exercises are extended with the underlying Python and algorithm patterns so you can adapt to unseen interview questions.

Study method Do not memorize definitions only. For each concept, explain the idea, work the example, name a failure mode, and state one trade-off. That pattern matches how stronger technical interviews are evaluated.

1. Complexity is part of the answer

For coding interviews, state time and space complexity and connect it to input scale. Big-O hides constants but communicates growth. A solution that scans once with a hash set is usually preferable to nested loops for large n , but memory constraints can change the choice.

Discuss worst-case and amortized behavior when relevant.

2. Lists, tuples, dictionaries and sets

Python lists are dynamic arrays with $O(1)$ amortized append and $O(n)$ insertion near the front. Tuples are immutable sequences. Dictionaries and sets are hash-table based and have average $O(1)$ lookup, insertion and deletion, subject to hashing behavior.

Choosing the right built-in data structure often removes most of the algorithmic complexity.

3. Two pointers and sliding windows

Two pointers exploit ordered or contiguous structure to avoid repeated scanning. Sliding windows maintain a summary of a moving range, which turns many $O(nk)$ computations into $O(n)$.

Always define what invariant the window maintains and when each pointer moves.

Worked example / interview scenario

For a moving average of width k , keep a running sum. Add the new value and subtract the value leaving the window instead of summing k values each time.

4. Hashing patterns

Frequency maps, membership sets and key-to-index maps solve many interview problems. Be explicit about duplicates, ordering and hashability. If stable first-seen order matters, modern Python dict preserves insertion order, but relying on that should be intentional.

```
def first_duplicate(xs):
    seen = set()
    for x in xs:
        if x in seen:
            return x
        seen.add(x)
    return None
```

5. Stacks, queues and monotonic structures

Stacks support nested structure, undo behavior and depth-first traversal. Queues support breadth-first traversal and streaming work. Use `collections.deque` for efficient append/pop from both ends. Monotonic stacks/queues maintain ordered candidates for next-greater or sliding-extreme problems.

Deep dive continued

6. Heaps and top-k streaming

A min-heap of size k tracks the largest k items in $O(n \log k)$ time and $O(k)$ memory. Python `heapq` is a min-heap. For the smallest k items, store directly; for largest k , keep a min-heap whose root is the smallest selected item.

Heaps are useful when sorting the entire dataset is unnecessary.

7. Trees and graphs

Tree recursion should define the base case and what each call returns. Graph traversal requires a visited structure to avoid cycles. BFS gives shortest path in an unweighted graph; Dijkstra handles non-negative weighted edges.

In Python, recursion depth can be a practical issue for very deep structures, so iterative traversal may be safer.

8. Generators and memory-efficient iteration

Generators yield values lazily instead of materializing all results. This is valuable for large datasets and streaming pipelines. An iterator has state and produces one item at a time; a generator function is a convenient way to implement an iterator.

In ML data loaders, generators can prevent loading an entire dataset into memory.

```
def batches(items, size):
    batch = []
    for x in items:
        batch.append(x)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
```

9. NumPy thinking: vectorization and shape discipline

NumPy operations run efficient compiled loops and make array shapes explicit. Vectorization can be much faster than Python loops, but temporary arrays can increase memory. Broadcasting is powerful but can silently produce wrong shapes if dimensions are misunderstood.

In interviews, explain both mathematical intent and array shapes.

10. Data leakage in Python preprocessing

Fit learned preprocessing only on training data. That includes scalers, imputers, target encoders, vocabulary builders and feature selection. Then apply the fitted transformation to validation/test data.

Pipelines help keep preprocessing and model fitting consistent and reduce accidental leakage.

Deep dive continued

11. Testing ML utility code

Test edge cases, empty inputs, duplicate values, numerical tolerance and invariants. For data transformations, assert schema, shape and missing-value behavior. Property-based thinking is useful: normalization should preserve row count; probabilities should stay in $[0,1]$.

Production ML code should be testable without requiring an expensive model run.

12. Concurrency and I/O intuition

Python threads are useful for I/O-bound work despite the GIL; CPU-bound Python code often benefits from multiprocessing or native vectorized libraries. Async I/O can scale many network-bound tasks but adds complexity.

For ML inference, bottlenecks are often in native/GPU code where the Python GIL is not the primary limit.

Complete case walkthrough

Case study - memory-bounded prediction-event processor

Exercise: build a memory-bounded stream processor that reads prediction events, deduplicates event IDs, keeps the top-10 anomaly scores and emits mini-batches for downstream storage.



Discuss memory limits, stable ordering, what happens when event IDs exceed dedupe memory, how to test partial batches, and the $O(\log k)$ update cost of the heap.

Additional interview drill

Answer each question aloud in 60-90 seconds. Then add one example, one failure mode and one trade-off.

#	Question	Strong answer should mention
1	List vs deque for queue?	deque gives efficient popleft
2	Why heap for top-k?	$O(n \log k)$ and $O(k)$ memory
3	BFS vs DFS?	shortest unweighted vs traversal/depth properties
4	Generator advantage?	lazy memory-efficient iteration
5	Dict average lookup?	$O(1)$ average hash table
6	What is amortized append?	occasional resize spread over many $O(1)$ appends
7	How avoid preprocessing leakage?	fit transformations on train only
8	Why vectorize NumPy?	compiled loops and lower Python overhead
9	How compute cosine safely?	dot divided by norms; handle zero vector
10	What tests for batch generator?	final partial batch, empty input, size validation
11	Why LRU cache?	bounded reuse with eviction of least recently used
12	Threads vs processes?	I/O vs CPU-bound Python, GIL considerations

Hands-on labs and worked exercises

These exercises turn the concepts into things you can calculate, code, debug or design. Work them before reading the expected reasoning.

Lab 1 - Implement top-k stream

Goal Use a heap to bound memory.

Steps

1. Process numbers one at a time.
2. Keep a min-heap of at most k values.
3. Push until full, then replace root only when incoming value is larger.
4. State $O(n \log k)$ time and $O(k)$ memory.

```
import heapq

def top_k(stream, k):
    h=[]
    for x in stream:
        if len(h)<k: heapq.heappush(h,x)
        elif x>h[0]: heapq.heapreplace(h,x)
    return sorted(h, reverse=True)
```

Expected reasoning / result

Sorting all n values costs $O(n \log n)$ and $O(n)$ storage; the heap is better when k is small.

Interview connection

Explain why the heap root is the smallest selected item.

Lab 2 - Sliding window average

Goal Remove repeated summation.

Steps

1. Compute first window sum.
2. For each step add entering value and subtract leaving value.
3. Emit sum/k.
4. State $O(n)$ time.

```
def moving_avg(xs, k):  
    if k <= 0 or k > len(xs): return []  
    s = sum(xs[:k]); out = [s/k]  
    for i in range(k, len(xs)):  
        s += xs[i] - xs[i-k]  
        out.append(s/k)  
    return out
```

Expected reasoning / result

Each element enters and leaves the running sum once.

Interview connection

Name the maintained invariant: s equals the sum of the current window.

Lab 3 - BFS shortest path

Goal Use a queue and visited set correctly.

Steps

1. Initialize queue with source and distance 0.
2. Pop oldest node.
3. Visit unseen neighbors and record parent/distance.
4. Stop when destination is found.

Expected reasoning / result

In an unweighted graph, BFS explores by distance layers, so the first visit gives the shortest edge count.

Interview connection

DFS does not guarantee shortest path in an unweighted graph.

Lab 4 - Leakage-safe scaler

Goal Separate fit and transform.

Steps

1. Compute mean/std only on training data.
2. Store parameters.
3. Transform validation/test with stored parameters.
4. Handle zero variance safely.

```
class ZScaler:
    def fit(self, x):
        import numpy as np
        self.mean=float(np.mean(x)); self.std=float(np.std(x)) or 1.0
        return self
    def transform(self, x):
        return [(v-self.mean)/self.std for v in x]
```

Expected reasoning / result

Calling fit on the full dataset leaks validation/test distribution into preprocessing.

Interview connection

The same principle applies to imputers, vocabularies and target encoding.

Lab 5 - Test a batch generator

Goal Practice edge-case thinking.

Steps

1. Test exact multiple of batch size.
2. Test final partial batch.
3. Test empty input.
4. Test invalid batch size.
5. Test that input order is preserved.

Expected reasoning / result

Interview code quality improves when you state assumptions and test boundaries.

Interview connection

A correct happy path is not enough for production Python.

Last-minute revision sheet

Use this page in the final 30-60 minutes before an interview. Say each answer aloud in one or two sentences; if you cannot, revisit the relevant chapter.

Topic	One-line recall
Two pointers	Use when sorted sequences let you decide the next output from current boundaries.
Stack	Use for nested structure or undo/backtracking-style local history.
BFS	Shortest path in unweighted graphs because exploration proceeds by distance layers.
Heap	Keep top-k or dynamic order statistics without sorting everything.
Generator	Produce one batch/item at a time to control memory.
Complexity	Always state time and auxiliary space after coding.
Edge cases	Empty input, duplicates, invalid values, ties, overflow/precision, streaming constraints.
ML twist	Separate fitting from transformation; avoid train-test contamination.

Research basis and further reading

This handbook is original LearnMLAcademy material. Public interview reports and official technical documentation were used to identify current topic emphasis and production practices. The explanations, examples, diagrams and code in this handbook are independently written.

Aced / Exponent - Meta Senior MLE Interview Experience (2026)

<https://www.tryexponent.com/experiences/meta-facebook-senior-machine-learning-engineer-interview-8719a7>

Candidate-reported Meta loop covering classic coding, AI-assisted debugging, behavioral questions and recommendation-system design with online/offline evaluation.

Amazon Jobs - SDE II Interview Prep

<https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>

Official guidance on syntactically correct code, edge cases, scalable solutions, Leadership Principles and STAR responses.

InterviewQuery - Google Data Scientist Interview Guide (2026)

<https://www.interviewquery.com/guides/google-data-scientist>

Recent candidate reports emphasize SQL window functions, A/B testing, product metrics, ML fundamentals and behavioral rounds.

Google Machine Learning Crash Course

Official ML learning material emphasizes practical exercises, production systems, testing and monitoring in addition to model theory.

Made With ML MLOps Course

Production-oriented curriculum covering data, modeling, testing, reproducibility, serving, CI/CD and monitoring.

Current 2026 interview-preparation resources

Public guides increasingly emphasize production trade-offs, system design, debugging and role-specific reasoning rather than definition-only question banks.

Keep learning at learnmlacademy.com

167 in-depth lessons • Python code examples • Visual diagrams • Projects • Interview preparation • Generative AI • LLM & RAG • Agentic AI • Always free